# Deep Tree Networks: A New Symbolic Deep Architecture

Zahra Ahmadi, Sina Malakouti, and Stefan Kramer

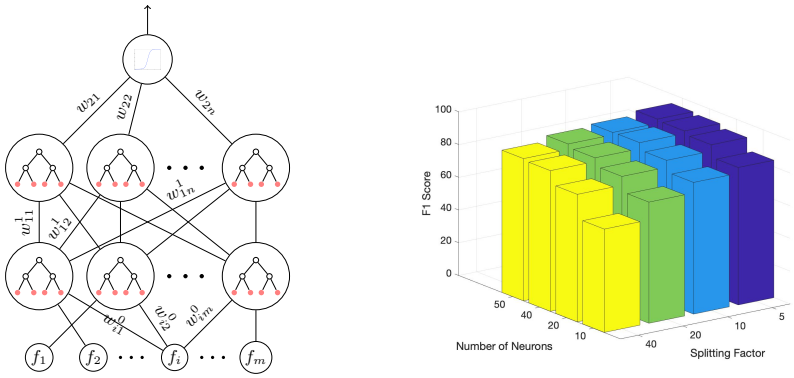
Institut für Informatik, Johannes Gutenberg-Universität, Mainz, Germany
zaahmadi,kramer@informatik.uni-mainz.de,s.malakouti2015@aut.ac.ir

**Abstract.** The recent success of deep neural networks has inspired many to wonder whether other learners could benefit from deeply layered architectures. In this paper, we investigate a novel deep architecture of Hoeffding trees. Our first experiments on MNIST show promising results in learning while keeping the network structure relatively small.

Although deep neural networks have shown their power by impressive results in several applications, they still suffer from a variety of problems. Deep neural networks are extremely complex models with many hyper-parameters. They need tremendous amounts of training data and powerful computational facilities to carefully tune the hyper-parameters, as the learning performance critically depends on a suitable choice. Even in the big data era, many tasks still lack sufficient amount of labeled data due to the high cost of labeling, leading to inferior performance of deep neural networks on those tasks. However, to exploit extensive training data, learning models should have a large capacity. Deep neural networks with their complex structure benefit from that compared to other learning models such as support vector machines. Another crucial ability of deep neural networks is representation learning. We aim to provide these properties for some other forms of learning (i.e., Hoeffding trees), while reducing the deficiencies of deep neural networks. Few other studies also combined decision trees with deep neural networks to accelerate training and reduce network complexity [1–3]; however, their strategy and model is different from our proposed method in many ways.

In the following, we explain our proposed method. Figure 1a illustrates an example snapshot of our proposed architecture. $w_{ij}^{l}$ is the weight of the network between the $i^{th}$ neuron in layer $l$ and the $j^{th}$ neuron in layer $l+1$, and $\{f_1, f_2, \ldots, f_m\}$ indicate instance features.

Different from linear activation functions in conventional deep networks, Deep Tree Networks (DTN) employ Hoeffding trees as their basic units. Hoeffding trees are well-performing decision trees that are capable of learning from massive data streams. They exploit the fact that a relatively small sample can often be enough to choose an optimal splitting attribute. However, decision trees are in general non-differentiable functions, and therefore they cannot be applied directly into updating formulas. To alleviate the issue, we use adaptive Naïve Bayes learners in the leaves of Hoeffding trees and convert them to piecewise differentiable functions. Deep Tree Networks densely connect all the layers except




(a) Snapshot of a DTN (b) Comparison of $F-$measure for different settings

Fig. 1: Snapshot of a Deep Tree Network (DTN) in our experiments (a), and grid search results on two network parameters: number of tree neurons per layer, and feature splitting factor for each input tree neuron (b).

the input. At the input layer, only a subset of features is passed to each neuron to add diversity in the following Hoeffding tree learners.

DTN training is composed of two phases: the forward pass, and the backward pass. In the forward pass, Hoeffding trees are built/updated, and in the backward pass the weights of the network are updated. In forward pass, first, the Hoeffding trees are built based on a random weighting on the subset of input features. Then, the estimated probability of the same training instances for a random class is passed to the next layer. For simplicity, we assume that the dataset is Gaussian ($\Phi = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-\frac{t^2}{2}} dt$), and we approximate $p(X = x) = \frac{d}{dx}\Phi \approx \Phi(x + \frac{\epsilon}{2}) - \Phi(x - \frac{\epsilon}{2})$.

All the posterior probabilities $P(C_j|X), j = \{1, \dots c\}$ are calculated and normalized to $N \sim (0, 1)$ for each instance to obtain a valid probability value. Subsequently, the input for the next layer is created by random selection of one predicted probability at neuron $i$ and fixing them for later batches and epochs. In the output layer, we use Softmax with the negative log-likelihood as loss function.

To update network weights, DTNs use Stochastic Gradient Descent (SGD). Assume that the instance $x$ at neuron $n$ ends in the leaf $l$. We only need to calculate the derivation of the Bayes model at node $l$: $\frac{dP_l(z)}{dz} = \frac{1}{\sqrt{2\pi}} e^{\frac{-z^2}{2}}$ , where $z = \frac{x-\mu}{\sigma}$. We normalize the results, and choose the random output based on previous selections and pass it to the previous layer. We use $l_2$ regularization.

## Experiments, Discussions, and Outlook

We conducted several experiments on MNIST to test the performance of DTN. We report the results for a network with two hidden layers to show the strength

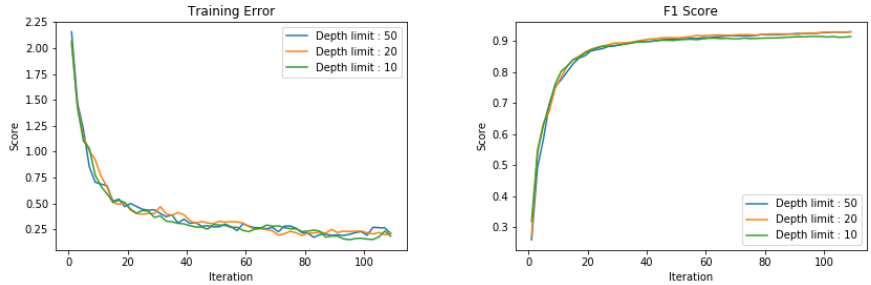



Fig. 2: Effect of Hoeffding tree depth on the performance of DTN.

of our model even without being extremely deep (Figure 1b). We chose the number of neurons in each hidden layer from $\{10, 20, 40, 50\}$ and the number of random input features from $\{\lfloor \frac{784}{40} \rfloor, \lfloor \frac{784}{20} \rfloor, \lfloor \frac{784}{10} \rfloor, \lfloor \frac{784}{5} \rfloor\}$, where 784 is the feature size of MNIST. Other parameters are fixed in all experiments: the number of epochs, batch size, SGD learning rate, the regularization term, and tree depth are 150, 100, 0.1, $10^{-4}$, and 20, respectively. Looking into the results for the F-measure for different settings, we observe that for a fixed number of neurons, providing more features to the first layer notably improves the performance. On the other hand, fixing the number of input features and expanding the network by its neuron size will improve the F-measure, but not as notable as having more overlap in features (e.g., F-measure changes from 63.08 to 83.94 in a network of 10 neurons when input features change from $\lfloor \frac{784}{40} \rfloor$ to $\lfloor \frac{784}{5} \rfloor$ vs. its change from 83.94 to 94.23 for $\lfloor \frac{784}{5} \rfloor$ input features with 10 and 50 neurons). We also changed the maximum depth of Hoeffding trees in each neuron to investigate its effect on the performance (Figure 2). Experiments for 50 neurons and $\lfloor \frac{784}{10} \rfloor$ input features indicate an unnoticeable difference in performance for depth values more than 10; however, for small values (e.g., maximum depth of 3 or 5), the F-measure degrades considerably.

Figure 1b shows the substantial effect of input features provided to the Hoeffding tree neurons. Hence, in our future experiments, we will develop models that distribute input features less random and allow Hoeffding tree neurons to work with higher-level features. We will also expand our experiments to other datasets.

## References

1. Gouk, H., Pfahringer, B., Frank, E.: Stochastic gradient trees. CoRR **abs/1901.07777** (2019)
2. Miller, K., Hettinger, C., Humpherys, J., Jarvis, T., Kartchner, D.: Forward thinking: Building deep random forests. CoRR **abs/1705.07366** (2017)
3. Zhou, Z., Feng, J.: Deep forest: Towards an alternative to deep neural networks. In: Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI). pp. 3553–3559 (2017)